

GitLab CI/CD Pipeline

Commands & Keywords Cheat Sheet

PIPELINE STRUCTURE KEYWORDS

stages:	Defines the order of pipeline stages	<i>e.g. [build, test, deploy]</i>
stage:	Assigns a job to a stage	<i>Job runs with others in same stage</i>
job-name:	Defines a job (any name you choose)	<i>Top-level key in .gitlab-ci.yml</i>
script:	Shell commands the job will run	<i>Required in every job</i>
before_script:	Runs before script in every job	<i>Good for setup / installs</i>
after_script:	Runs after script, even on failure	<i>Good for cleanup</i>
include:	Import another YAML file into pipeline	<i>Reuse config across projects</i>
extends:	Inherit config from another job	<i>Like copy-paste but smarter</i>

COMMON SCRIPT COMMANDS

echo "message"	Print a message to the job log	<i>Great for debugging</i>
cd /path/to/dir	Change into a directory	
ls -la	List files in current directory	<i>Useful to verify files exist</i>
mkdir -p folder/	Create a folder (no error if exists)	<i>-p creates parent dirs too</i>
cp source dest	Copy a file	
mv source dest	Move or rename a file	
rm -rf folder/	Delete a file or folder	<i>Be careful — no undo!</i>
cat file.txt	Print contents of a file	
export VAR=value	Set an environment variable	<i>Available to all commands after</i>
chmod +x script.sh	Make a script executable	<i>Run it with ./script.sh after</i>
curl -o file URL	Download a file from a URL	
apt-get install pkg	Install a package (Debian/Ubuntu)	<i>Add -y to skip confirmation</i>
pip install pkg	Install a Python package	
npm install	Install Node.js dependencies	<i>Reads from package.json</i>

CONTROL & CONDITIONS

rules:	Conditionally include/exclude a job	<i>Replaces only: / except:</i>
- if: \$VAR == "value"	Run job only when condition is true	<i>Uses CI variables</i>
- when: manual	Job needs a human click to run	<i>Shows play button in UI</i>
- when: always	Always run this job	<i>Even if previous jobs failed</i>
- when: on_failure	Only run if a previous job failed	<i>Good for alerts/cleanup</i>
- when: never	Never run this job	<i>Effectively disables it</i>
allow_failure: true	Pipeline stays green even if job fails	
needs:	Run job as soon as listed jobs finish	<i>Skips waiting for whole stage</i>
dependencies:	Download artifacts from specific jobs	<i>Limits what gets downloaded</i>
timeout:	Kill job if it runs too long	<i>e.g. timeout: 30 minutes</i>
retry:	Auto-retry a failed job N times	<i>e.g. retry: 2</i>

VARIABLES, ARTIFACTS & CACHE

variables:	Define custom variables for the job	<i>Accessible as \$VAR_NAME</i>
\$CI_COMMIT_BRANCH	Name of the current branch	<i>Built-in GitLab variable</i>
\$CI_JOB_ID	Unique ID of this job run	<i>Built-in GitLab variable</i>
\$CI_PROJECT_NAME	Name of the project	<i>Built-in GitLab variable</i>
\$CI_COMMIT_SHA	Full commit hash	<i>Built-in GitLab variable</i>
artifacts:	Save files after the job finishes	<i>Other jobs can download them</i>
paths:	Which files/folders to save	<i>e.g. paths: [dist/, report.html]</i>
expire_in:	How long to keep artifacts	<i>e.g. expire_in: 1 week</i>
reports:	Special artifacts GitLab understands	<i>e.g. junit, coverage, sast</i>
cache:	Save files between pipeline runs	<i>Speeds up installs</i>
key:	Name for this cache bucket	<i>Same key = shared cache</i>
paths:	Which folders to cache	<i>e.g. paths: [node_modules/]</i>

RUNNER, ENVIRONMENT & IMAGES

image:	Docker image to run the job inside	<i>e.g. image: python:3.11</i>
services:	Extra containers (e.g. a database)	<i>e.g. services: [postgres:14]</i>
tags:	Pick which Runner handles the job	<i>Must match tag on registered Runner</i>
environment:	Link job to a GitLab environment	<i>Used for deployments / tracking</i>
name:	Environment name (e.g. production)	
url:	URL shown in GitLab UI after deploy	
parallel:	Run N copies of the job at once	<i>e.g. parallel: 5</i>
interruptible: true	Cancel job if a newer pipeline starts	<i>Saves runner minutes</i>

MINIMAL PIPELINE EXAMPLE

```
stages: - build - test - deploy
build-job: stage: build image: node:18 script: - echo "Installing dependencies..." - npm install - npm run build artifacts: paths: [dist/]
test-job: stage: test image: node:18 script: - npm test tags: - docker
deploy-job: stage: deploy script: - echo "Deploying to production!"
rules: - if: $CI_COMMIT_BRANCH == "main" when: manual
```

Tip: Your pipeline file is always named `.gitlab-ci.yml` and lives in the root of your repo.

docs.gitlab.com/ee/ci